

UNIDAD 2

OBJETOS, CLASES, HERENCIA Y POLIMORFISMO.

Tema:



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

**FUNDAMENTOS DE CONSTRUCTORES
Y DESTRUCTORES**

GUÍA DE ESTUDIO

SEMANA 7

ÍNDICE

1.1. INTRODUCCIÓN.....	1
1.2. RESULTADO DE APRENDIZAJE	2
1.3. RUTA DE APRENDIZAJE	2
1.4. DESARROLLO DE CONTENIDOS.....	3
1.4.1 Del ciclo de vida del objeto a la programación moderna en Python.....	3
1.4.2 Recordando la arquitectura del proyecto Biblioteca.....	3
1.4.3 El constructor <code>__init__</code>	4
1.4.4 Constructor aplicado a la clase Libro.....	5
1.4.5 Del <code>main.py</code> al constructor y al servicio Biblioteca	6
1.4.6 Parámetros y valores predeterminados.....	6
1.4.7 Control de atributos con <code>@property</code>	7
1.4.8 Actualización de datos con <code>@setter</code>	8
1.4.9 Python moderno: <code>@dataclass</code>	9
1.4.10 Cuándo usar <code>__init__</code> y cuándo <code>@dataclass</code>	9
1.4.11 El destructor <code>__del__</code> y su uso actual	10
1.4.12 Constructor y destructor en el ciclo de vida del objeto.....	11
1.4.13 Repositorio docente y estructura modular	12
1.4.14 Fragmentos esenciales del código docente.....	12
1.4.15 Buenas prácticas aplicadas.....	15
1.5. CONCLUSIONES	15
1.6. BIBLIOGRAFÍA.....	16

1.1. INTRODUCCIÓN

La Semana 7 continúa el desarrollo de la Unidad 2, denominada Objetos, clases, herencia y polimorfismo. En las semanas anteriores se trabajó la organización modular de proyectos en Python, la definición de clases y objetos, el uso de atributos y métodos, la separación entre modelos, servicios y archivo principal, así como la aplicación progresiva de buenas prácticas en la construcción de un sistema de biblioteca.

En esta semana, el estudio se concentra en los constructores y destructores. El constructor permite inicializar correctamente un objeto desde el momento en que se crea, mientras que el destructor forma parte del ciclo de vida del objeto y debe comprenderse como un mecanismo de limpieza para escenarios específicos. El énfasis principal de la guía está en el uso correcto de `__init__`, `@property`, `@setter` y `@dataclass` dentro de un proyecto Python moderno.

El caso de estudio continúa siendo un sistema de gestión de biblioteca. En esta versión, el proyecto trabaja con las clases Libro, Usuario y Biblioteca. La clase Libro utiliza un constructor tradicional con validaciones mediante propiedades. La clase Usuario utiliza `@dataclass` para mostrar una forma moderna de generar automáticamente el constructor cuando una clase está orientada principalmente al almacenamiento de datos.

La guía complementa las diapositivas y el repositorio docente de la Semana 7. Por esta razón, no se copia todo el código fuente dentro del documento. En su lugar, se presentan fragmentos esenciales que permiten comprender cómo se aplican los conceptos más importantes. El código completo debe revisarse en el repositorio correspondiente y ejecutarse durante la práctica en clase.

1.2. RESULTADO DE APRENDIZAJE

Implementa programas que hacen uso de relación de herencia, interfaces y clases abstractas con métodos y variables polimórficas.

1.3. RUTA DE APRENDIZAJE

La ruta de aprendizaje de la Semana 7 integra la revisión conceptual con la aplicación práctica en el proyecto Biblioteca. Primero se analiza el papel del constructor en la creación de objetos. Luego se estudia el control de atributos mediante `@property` y `@setter`. Después se introduce `@dataclass` como recurso moderno de Python. Finalmente, se revisa el destructor desde una perspectiva actual, diferenciando su valor conceptual de su uso real en aplicaciones modernas.

COMPONENTE	ACTIVIDAD	CALIFICACIÓN
Componente en contacto con el docente	Cuestionario sobre constructores, destructores, <code>@property</code> , <code>@setter</code> y <code>@dataclass</code> .	10 puntos
Componente práctico-experimental	Taller práctico de aplicación de constructores, propiedades y arquitectura modular en Python.	10 puntos
Componente autónomo	Actividad de refuerzo sobre el ciclo de vida del objeto y buenas prácticas en Python moderno.	10 puntos

Las actividades buscan que el estudiante comprenda que un objeto no solo debe existir dentro del programa, sino que debe nacer con un estado válido, mantener sus datos controlados y formar parte de una estructura organizada de código.

1.4. DESARROLLO DE CONTENIDOS

1.4.1 Del ciclo de vida del objeto a la programación moderna en Python

En Programación Orientada a Objetos, una clase define la estructura general de una entidad, pero el objeto es la instancia concreta que se utiliza durante la ejecución del programa. Para que ese objeto funcione correctamente, debe iniciar con datos válidos y coherentes. Esta responsabilidad recae principalmente en el constructor.

La Semana 7 responde una pregunta central: ¿cómo garantizamos que cada objeto nazca correctamente inicializado antes de comenzar a trabajar con él? A partir de esta pregunta se estudian los constructores, las propiedades, los métodos setter, el decorador `@dataclass` y el destructor como parte del ciclo de vida del objeto.

Concepto	Función dentro de la Semana 7
Constructor <code>__init__</code>	Inicializa los atributos necesarios cuando se crea un objeto.
<code>@property</code>	Permite consultar un atributo de forma controlada sin romper la forma natural de uso del objeto.
<code>@setter</code>	Permite modificar un atributo aplicando reglas o validaciones.
<code>@dataclass</code>	Genera automáticamente el constructor y otros métodos útiles en clases orientadas a datos.
Destructor <code>__del__</code>	Permite comprender la etapa final del ciclo de vida del objeto; su uso directo en aplicaciones modernas es poco frecuente.

1.4.2 Recordando la arquitectura del proyecto Biblioteca

El proyecto mantiene la arquitectura modular trabajada en semanas anteriores. Esta organización separa las entidades del sistema, la lógica de gestión y el punto de entrada del programa. La carpeta `modelos` contiene las clases principales; la carpeta `servicios` administra las operaciones; y `main.py` coordina la interacción con el usuario mediante un menú de consola.

```
biblioteca_app/  
|  
├─ modelos/  
|   ├── __init__.py  
|   ├── libro.py  
|   └─ usuario.py  
|  
├─ servicios/  
|   ├── __init__.py  
|   └─ biblioteca.py  
|  
└─ main.py
```

Nota. El constructor se ubica principalmente dentro de las clases de modelo, porque allí se inicializan los atributos que describen a cada objeto.

1.4.3 El constructor `__init__`

El constructor `__init__` es un método especial que se ejecuta automáticamente cuando se crea una instancia de una clase. Su responsabilidad principal es inicializar los atributos del objeto con los valores recibidos. De esta manera, el objeto queda listo para ser utilizado por el resto del programa.

En Python, el constructor no se llama manualmente. Cuando se escribe `Libro("Python", "Autor", "ISBN")`, Python crea el objeto y ejecuta `__init__` para asignar los valores iniciales. Este proceso permite que cada objeto tenga su propia información aunque todos pertenezcan a la misma clase.

```
class Libro:  
  
    def __init__(self, titulo: str, autor: str, isbn: str, disponible: bool = True):  
        self.titulo = titulo  
        self.autor = autor  
        self.isbn = isbn  
        self.disponible = disponible
```

En el fragmento anterior, Libro es la clase. Dentro del constructor se definen los atributos que tendrá cada objeto creado a partir de esa clase. Cada vez que se cree un nuevo libro, esos atributos podrán almacenar valores diferentes.

Elemento	Función
__init__	Inicializa el objeto al momento de crearlo.
self	Representa la instancia actual del objeto.
título, autor, isbn	Parámetros que recibe el constructor.
disponible=True	Valor predeterminado que permite crear el objeto como disponible si no se indica otro estado.

1.4.4 Constructor aplicado a la clase Libro

En el sistema Biblioteca, la clase Libro representa una entidad principal. Cada libro debe iniciar con título, autor, ISBN y disponibilidad. El constructor permite que estos datos se asignen desde el momento en que el objeto se crea.

La implementación de la clase Libro utiliza el constructor tradicional porque se desea tener mayor control sobre los atributos. Además, los atributos se asignan mediante propiedades, lo que permite aplicar validaciones cuando se recibe información del usuario.

```
libro = Libro(  
    "Python desde Cero",  
    "Autor Ejemplo",  
    "9780000000001"  
)
```

El objeto libro nace con sus datos iniciales. Luego puede ser enviado al servicio Biblioteca para registrarlo, listarlo o buscarlo dentro de la colección de libros.

Nota. En este proyecto, el constructor tradicional se utiliza como base principal porque permite observar de forma explícita cómo se inicializa cada atributo del objeto.

1.4.5 Del main.py al constructor y al servicio Biblioteca

El constructor no trabaja de forma aislada. En el proyecto, la creación de objetos inicia desde main.py. Cuando el usuario registra un libro, el programa solicita los datos, crea una instancia de Libro y luego entrega ese objeto al servicio Biblioteca para almacenarlo.

```
def registrar_libro(biblioteca):  
    print("\n--- Registrar libro ---")  
  
    titulo = input("Título: ")  
    autor = input("Autor: ")  
    isbn = input("ISBN: ")  
  
    libro = Libro(titulo, autor, isbn)  
    biblioteca.agregar_libro(libro)  
  
    print("\nLibro registrado correctamente.")
```

Este flujo permite observar la relación entre las capas del proyecto. main.py recibe los datos, el modelo Libro construye el objeto y el servicio Biblioteca administra la colección de objetos registrados.

Paso	Acción
1	main.py solicita los datos al usuario.
2	Se crea el objeto Libro con los datos ingresados.
3	Python ejecuta automáticamente el constructor <code>__init__</code> .
4	El objeto queda inicializado.
5	Biblioteca.agregar_libro() almacena el objeto en la lista de libros.

1.4.6 Parámetros y valores predeterminados

Los constructores pueden recibir parámetros obligatorios y parámetros con valores predeterminados. Los parámetros obligatorios deben enviarse al momento de crear el objeto. Los valores predeterminados, en cambio, permiten crear objetos más flexibles cuando cierta información puede asumir un valor inicial común.

En la clase Libro, titulo, autor e isbn son datos necesarios. El atributo disponible tiene el valor predeterminado True, porque normalmente un libro nuevo se registra como disponible.

```
def __init__(self, titulo: str, autor: str, isbn: str, disponible: bool = True):
    self.titulo = titulo
    self.autor = autor
    self.isbn = isbn
    self.disponible = disponible
```

Tipo de parámetro		Ejemplo	Interpretación
Obligatorio	titulo		Debe proporcionarse para crear el objeto.
Obligatorio	autor		Debe proporcionarse para identificar el autor del libro.
Obligatorio	isbn		Debe proporcionarse como identificador bibliográfico.
Predeterminado	disponible=True		Si no se indica otro valor, el libro se crea como disponible.

1.4.7 Control de atributos con @property

El decorador @property permite acceder a un método como si fuera un atributo. Esta técnica se utiliza para controlar la lectura de datos internos sin obligar al estudiante a utilizar métodos tradicionales como obtener_titulo().

En la clase Libro, el atributo interno _titulo almacena el valor real. La propiedad titulo permite consultarlo de forma natural mediante libro.titulo, manteniendo la posibilidad de controlar internamente cómo se accede al dato.

```
@property
def titulo(self) -> str:
    return self._titulo
print(libro.titulo)
```

Aunque visualmente parece que se accede a un atributo, internamente Python está ejecutando el método decorado con `@property`. Esta práctica permite mantener una interfaz sencilla para el usuario del objeto y, al mismo tiempo, conservar control sobre la información interna.

Nota. *`@property` complementa el principio de encapsulación, porque permite consultar datos de manera controlada sin romper la forma natural de trabajar con objetos en Python.*

1.4.8 Actualización de datos con `@setter`

El decorador `@setter` permite controlar la modificación de un atributo. En lugar de permitir que cualquier valor sea asignado directamente, el setter puede validar el dato antes de almacenarlo en el objeto.

En la clase `Libro`, el setter de `titulo` verifica que el nuevo título no esté vacío. Si el valor recibido no cumple la condición, se genera un `ValueError`. Esta validación evita que el objeto quede en un estado inválido.

```
@titulo.setter
def titulo(self, nuevo_titulo: str):
    if not nuevo_titulo.strip():
        raise ValueError("El título no puede estar vacío.")

    self._titulo = nuevo_titulo
```

Intento de asignación	Resultado
<code>libro.titulo = "Python avanzado"</code>	El nuevo título se almacena correctamente.
<code>libro.titulo = ""</code>	Se genera un error porque el título no puede estar vacío.

El setter ayuda a proteger el estado interno del objeto. En este sentido, no solo permite modificar datos, sino que también define reglas para mantener la consistencia de la información.

1.4.9 Python moderno: @dataclass

Python ofrece herramientas modernas que reducen código repetitivo. Una de ellas es `@dataclass`, un decorador que permite declarar clases orientadas principalmente al almacenamiento de datos. Al utilizar `@dataclass`, Python genera automáticamente el constructor y otros métodos útiles.

En el proyecto de la Semana 7, la clase `Usuario` utiliza `@dataclass` porque su propósito principal es almacenar nombre, correo e identificador. Esto permite comparar el constructor tradicional utilizado en Libro con una forma más compacta y moderna de definir una clase de datos.

```
from dataclasses import dataclass

@dataclass
class Usuario:
    nombre: str
    correo: str
    id_usuario: int
```

Gracias a esta definición, ya no es necesario escribir manualmente el método `__init__`. Python lo genera automáticamente a partir de los atributos declarados en la clase.

```
usuario = Usuario("Juan Pérez", "juan@correo.com", 1)
```

Nota. `@dataclass` no elimina la importancia de comprender `__init__`. Primero se debe entender el constructor tradicional; luego se puede aprovechar `@dataclass` cuando la clase se orienta principalmente a representar datos.

1.4.10 Cuándo usar `__init__` y cuándo `@dataclass`

Tanto `__init__` como `@dataclass` son opciones válidas. La elección depende del propósito de la clase. Si la clase requiere mayor control, validaciones personalizadas o lógica

específica de inicialización, conviene utilizar el constructor tradicional. Si la clase representa principalmente datos y no requiere una inicialización compleja, `@dataclass` puede simplificar el código.

Constructor tradicional <code>__init__</code>	Decorador <code>@dataclass</code>
Ofrece mayor control sobre la inicialización.	Reduce código repetitivo.
Permite escribir validaciones desde el inicio.	Genera automáticamente el constructor.
Es adecuado para clases con lógica personalizada.	Es adecuado para clases orientadas principalmente a datos.
Ayuda a comprender la base conceptual de la POO.	Es una herramienta moderna que mejora la claridad cuando se usa correctamente.

En esta semana se utiliza `__init__` como fundamento principal y `@dataclass` como alternativa moderna. Esta combinación permite que el estudiante comprenda el funcionamiento interno y, al mismo tiempo, conozca prácticas actuales de programación en Python

1.4.11 El destructor `__del__` y su uso actual

El destructor `__del__` es un método especial que puede ejecutarse cuando un objeto está por ser eliminado. En Python, este método existe como parte del ciclo de vida del objeto, pero su uso directo en aplicaciones modernas es poco frecuente.

La razón principal es que Python administra automáticamente la memoria mediante el recolector de basura. Además, para liberar recursos como archivos, conexiones o sesiones, actualmente se prefieren mecanismos más seguros y predecibles, como administradores de contexto `with`, métodos explícitos `close()` o herramientas propias de los frameworks.

```
class RecursoTemporal:

    def __init__(self):
        print("Recurso creado.")

    def __del__(self):
        print("Recurso liberado.")
```

```
recurso = RecursoTemporal()  
del recurso
```

Contexto	Uso actual del destructor
Aplicaciones comunes en Python	Su uso directo es poco frecuente.
Archivos	Se recomienda with open(...) en lugar de __del__.
Bases de datos y frameworks	Se usan sesiones, close(), context managers o mecanismos del framework.
Librerías especializadas	Puede aparecer en integración con C/C++, memoria nativa, GPU, hardware o recursos del sistema.

Nota. El destructor debe comprenderse para reconocer el ciclo de vida del objeto, pero en aplicaciones como el sistema Biblioteca no se recomienda usarlo para cerrar archivos o controlar la lógica principal del programa.

1.4.12 Constructor y destructor en el ciclo de vida del objeto

El ciclo de vida de un objeto permite comprender cuándo nace, cómo se utiliza y cuándo deja de formar parte activa del programa. En esta semana, el constructor representa el inicio del objeto y el destructor representa la etapa final, aunque su uso directo se limita a situaciones específicas.

```
Clase  
↓  
Constructor __init__  
↓  
Objeto inicializado  
↓  
Métodos y propiedades  
↓  
Objeto deja de utilizarse
```



Destructor `__del__` (solo en casos específicos)

En el proyecto Biblioteca, el aprendizaje principal se concentra en el constructor, las propiedades, los setters y `@dataclass`. El destructor se estudia para comprender el ciclo de vida del objeto y para diferenciar entre una característica del lenguaje y una práctica habitual de desarrollo.

1.4.13 Repositorio docente y estructura modular

Como complemento a la presente guía, el estudiante debe revisar el repositorio docente de la Semana 7. Este recurso contiene la implementación completa del sistema Biblioteca y permite ejecutar el código trabajado para la clase.

Enlace al repositorio docente: <https://github.com/kevin10lascano-sketch/Clase-Semana-7-POO.git>

1.4.14 Fragmentos esenciales del código docente

En esta sección se presentan los fragmentos más importantes del código docente. El objetivo es que el estudiante identifique cómo se aplican los conceptos centrales de la semana sin copiar todo el proyecto dentro de la guía.

Clase Libro: constructor tradicional, propiedades y validaciones

```
class Libro:

    def __init__(self, titulo: str, autor: str, isbn: str, disponible: bool = True):
        self.titulo = titulo
        self.autor = autor
        self.isbn = isbn
        self.disponible = disponible
```

La clase Libro utiliza el constructor tradicional porque requiere control sobre la inicialización y validación de atributos. Al asignar `self.titulo`, `self.autor` y `self.isbn`, se activan los setters correspondientes.

```
@property
def isbn(self) -> str:
    return self._isbn

@isbn.setter
def isbn(self, nuevo_isbn: str):
    if not nuevo_isbn.strip():
        raise ValueError("El ISBN no puede estar vacío.")

    self._isbn = nuevo_isbn
```

La propiedad `isbn` permite consultar el valor de manera natural, mientras que el setter evita que se almacene un ISBN vacío.

```
def mostrar_informacion(self) -> str:
    estado = "Disponible" if self.disponible else "Prestado"

    return (
        f"Título: {self.titulo} | "
        f"Autor: {self.autor} | "
        f"ISBN: {self.isbn} | "
        f"Estado: {estado}"
    )
```

El método `mostrar_informacion()` devuelve una representación legible del objeto. Esto mantiene organizada la forma en que cada entidad presenta sus datos.

Clase Usuario: uso de `@dataclass`

```
@dataclass
class Usuario:
    nombre: str
    correo: str
```

```

id_usuario: int

def mostrar_informacion(self) -> str:
    return (
        f"ID: {self.id_usuario} | "
        f"Nombre: {self.nombre} | "
        f"Correo: {self.correo}"
    )

```

La clase Usuario demuestra el uso de `@dataclass`. Python genera automáticamente el constructor con los atributos nombre, correo e id_usuario.

Clase Biblioteca: gestión de libros y usuarios

```

class Biblioteca:

    def __init__(self):
        self.libros = []
        self.usuarios = []

    def agregar_libro(self, libro: Libro):
        self.libros.append(libro)

    def agregar_usuario(self, usuario: Usuario):
        self.usuarios.append(usuario)

```

La clase Biblioteca pertenece a la capa de servicios. No representa un libro ni un usuario específico, sino que administra colecciones de objetos creados desde los modelos.

```

def buscar_libro(self, titulo: str):
    for libro in self.libros:
        if libro.titulo.lower() == titulo.lower():
            return libro

    return None

```

El método `buscar_libro()` recorre la lista de libros y compara el título ingresado con el título de cada objeto registrado.

1.4.15 Buenas prácticas aplicadas

Buena práctica	Aplicación en la Semana 7	Beneficio
Arquitectura modular	Uso de modelos, servicios y main.py.	Organiza responsabilidades y facilita el mantenimiento.
Constructor claro	Libro inicializa sus atributos mediante <code>__init__</code> .	Permite que el objeto nazca con datos completos.
Propiedades	<code>@property</code> permite consultar atributos controlados.	Mantiene una interfaz natural para acceder a los datos.
Setters	<code>@setter</code> valida cambios antes de almacenarlos.	Evita estados inválidos dentro del objeto.
Uso moderno de Python	Usuario se define con <code>@dataclass</code> .	Reduce código repetitivo en clases orientadas a datos.
Métodos pequeños	main.py divide el menú en funciones específicas.	Facilita la lectura y la depuración.
Destructor con enfoque conceptual	<code>__del__</code> se estudia como parte del ciclo de vida del objeto.	Evita enseñar prácticas forzadas o poco recomendadas para aplicaciones modernas.

Estas prácticas permiten que el estudiante comprenda que programar no consiste únicamente en lograr que el código ejecute. Un programa debe ser claro, mantenible y coherente con las herramientas actuales del lenguaje.

Nota. En el taller práctico, el estudiante debe aplicar constructores, propiedades y organización modular. El destructor debe entenderse conceptualmente y no utilizarse de forma forzada para resolver tareas que Python moderno maneja mejor con otros mecanismos.

1.5. CONCLUSIONES

El constructor `__init__` inicializa los atributos necesarios para que un objeto comience a trabajar correctamente dentro del programa.

`@property` y `@setter` permiten controlar el acceso y la modificación de ciertos atributos, fortaleciendo la encapsulación en Python moderno.

`@dataclass` representa una alternativa moderna para crear clases orientadas a datos de forma más compacta, sin perder claridad en la estructura del proyecto.

El destructor `__del__` debe comprenderse como parte del ciclo de vida del objeto y como un mecanismo útil en casos específicos, no como una herramienta obligatoria para todas las clases.

El proyecto Biblioteca permite integrar estos conceptos dentro de una arquitectura modular sencilla, preparando al estudiante para desarrollar aplicaciones más organizadas y mantenibles.

1.6. BIBLIOGRAFÍA

Boucheny, V. (2025). Aprender programación orientada a objetos con Python (3.^a ed.). Ediciones ENI.

Downey, A. (2024). Think Python (3rd ed.). O'Reilly Media.

Hernández Bejarano, M., & Baquero Rey, L. E. (2023). Programación orientada a objetos en Java: Buenas prácticas. Ediciones de la U.

Hernández Bejarano, M., & Baquero Rey, L. E. (2025). Python con orientación a objetos y al análisis de datos. Ediciones de la U.

Piñeiro Gómez, J. M. (2022). Principios de la programación orientada a objetos. Paraninfo.